

Where we are standing

A backtester that reports an estimate, its error bar, and the probability the number survived the search that produced it.

Commit	92e0748 on main · 19 commits	Suite	183 collected · 179 pass · 4 skip · 46.8 s
Latest	docs: gate status through G5, refreshed status report	CI	green · 5 checks · offline, no secrets
Code	~1,700 package · ~2,900 test LOC	Quality	ruff clean · mypy --strict clean (31 files)

Gate 0.0, G1, G2, G3, G4, G5 and G7 are green — the entire certified offline core. No market data has entered the system, so invariant B1 still holds, and everything to date runs in CI with no network, no API key and no rate limit. G6 is next and is the first gate that needs the data layer.

This session closed on a report that G2, G3 and G4 were failing. Each was stressed far past its own gate rather than re-running the gate that had already passed. G2 and G5 were and remain green; G3 carried a real defect, now fixed; G4 was correct but under-covered. The stress harness is committed as scripts/health_check.py, and the full investigation is in docs/logs/2026-08-08-health-check.md.

Health check — stressing the gates past their own fixtures

A gate can be green because the property holds, or green because the fixture happened to be kind. These are the same properties over a wide parameter grid, which is a different question and the one worth asking when a result is doubted.

Check	Coverage	Result
G2 twin engines	1,890 combinations, all six Result fields	worst 0.000e+00
G4 zero-cost identity	288 engine x convention x process x length x seed	288/288 bitwise
G3 recovery	800 paths vs exact lognormal targets	worst 0.90 SE
G3 paired invariant	simple SR - log SR must equal sigma/2 exactly	0.00 SE
G5 monotonicity	3 seeds x 4 strategies, 0-100 bps	12/12 monotone
Invariants B7-B10	frozen types, annualisation, seeds, kurtosis, 0.4	0 violations

The one real defect. G3's targets were first-order approximations. A simple return is $\exp(g) - 1$ for normal g , so it is lognormal and its exact Sharpe is 0.399921 rather than $\mu/\sigma = 0.400000$, and its exact volatility 0.200071 rather than $\sigma = 0.200000$ — σ is the volatility of the *log* returns. Errors of 0.02% and 0.04%, immaterial against sampling noise, but a gate whose whole job is known-truth recovery should not carry an approximation. Both are now exact.

The one real coverage gap. G4 only ever exercised $w = 1$, and 1 is a fixed point of most weight bugs: a wrong sign, a double-scaling by exposure, or a one-bar misalignment at fractional exposure all leave it untouched. An identity holding only at $w = 1$ is not an identity. 33 tests now cover w in $\{1, \frac{1}{2}, \frac{1}{4}, -\frac{1}{2}, -1\}$ across both engines and all conventions, plus a check that the short leg pays borrow and a long leg is not charged it.

No bias was found. The reported discrepancy was sampling noise: the absolute error falls 47× between 200 and 3,000 paths, which is $1/\sqrt{M}$ convergence. The paired invariant is the durable protection — because it is measured on the same paths the noise cancels, its standard error is 2.6% of the level standard error, and swapping the two Sharpe conventions fails it even when both level tests still pass.

Gate status

Gate	Statement	Status	Evidence
0.0	Reproduce the propositions first	GREEN	$E[\max z]$ to 0.58 SE; Prop 3 slope -0.9989 ± 0.0014
G1	Causality: scramble future, past bit-identical	GREEN	500 cuts × 20 seeds per strategy, 3 strategies
G2	Twin engines agree to 1e-12	GREEN	0.000e+00 across zoo × 3 conventions × cost sweep

Gate	Statement	Status	Evidence
G3	Analytic recovery on synthetic GBM	GREEN	both Sharpe conventions, vol, drift; $1/\sqrt{M}$; power test
G4	Zero-cost identity	GREEN	bitwise, both engines, all three conventions
G5	Cost monotonicity, break-even cost	GREEN	monotone 0-100 bps; $c^* = 27.03$ bps per turn
G6	Null calibration, 1,000 coin flips	not started	next; first gate needing the data layer
G7	Leakage trap must fire	GREEN	5 traps rejected; A4 oracle case ruled on
G8	Purged, embargoed walk-forward	not started	deferrable per 03 Part C
G9	PBO via CSCV	partial	SelectionRule interface built first, by design
G10	Reproducibility from pinned hashes	partial	figure bytes + numeric output stable across runs

G2 — the session's result

Two engines, written independently, sharing exactly one thing: the warm-up index. That is shared deliberately, because an off-by-one in the warm-up is a specification question rather than an implementation one, and duplicating it would let both engines be wrong in the same direction while agreeing perfectly. All of the Part E arithmetic is written twice on purpose — two engines that share their accounting agree by construction and certify nothing.

Measurement	Value	Reading
Max relative equity deviation	0.000e+00	Exact, against a $1e-12$ requirement
Coverage	3 strategies $\times$ 3 conventions $\times$ 5 cost models	Plus hypothesis over arbitrary price series
Planted engine disagreement	$7.417e-02$	Gate discriminates; it is not passing vacuously
Convention spread (final equity)	12924.37 / 12915.83 / 11688.41	close_to_close / next_open / next_close

Exact agreement is legitimate here rather than suspicious: the event engine's hard-sliced signal window contains the same elements as the vectorised engine's rolling window, and the arithmetic order coincides. The planted-bug test is what proves the harness can fail. The equity path is an explicit loop in *both* engines because it is not a cumprod — $\text{cost}[t]$ depends on $\text{equity}[t-1]$, and Part F2's approximate route carries error of order cost^2 that would never survive $1e-12$.

G3 and G5 — recovery and the cost of turnover

Quantity	Measured	True value	Gap
Simple-return Sharpe, 200 paths	+0.37513 +/- 0.02358	$\mu/\sigma = 0.40$	1.05 SE
Log-return Sharpe, 200 paths	+0.27461 +/- 0.02364	$(\mu - s^2/2)/\sigma = 0.30$	1.07 SE
Annualised volatility	0.20018 +/- 0.00020	$\sigma = 0.20$	0.09% rel
Annualised log drift	+0.05502 +/- 0.00474	$\mu - s^2/2 = 0.06$	1.05 SE
Monte Carlo SE scaling	slope -0.5173, $r^2 = 0.997$	-0.5 exactly	confirms $1/\sqrt{M}$
Mean reversion on GBM	+0.0051 +/- 0.0738	0 -- no edge exists	invents none
Mean reversion on AR(1), $\phi=0.95$	+1.1534 +/- 0.0703	> 0 -- edge exists	16 SE, finds it
Break-even cost c^*	27.03 bps / turn	at 68.8 turns/yr	+0.35 SR at $c^*/2$

The last two rows are the pair that matters and the one people skip. 0.1 shows the engine does not invent edge; 0.3 shows it does not destroy edge that exists. A framework failing to find signal in a series that provably contains signal is broken in a way no real-data test reveals, because on real data “found nothing” is always a plausible answer.

What the gates caught

Every row is a real defect in code I had written, found by a check rather than by reading. This is the argument for the whole approach, so it is worth keeping the score honestly.

Found by	Defect	Why it was dangerous
hypothesis	Constant column escaped the degeneracy check	60 copies of 0.001 give $\text{std} \approx 2\text{e-}19$, not 0.0, so an equality test passed it through and the Sharpe came out near 1e16 — enough to take the entire portfolio under ArgMax or Softmax
hypothesis	Softmax amplified rounding dust into a decision	Columns tied on true Sharpe leave $\sim 1\text{e-}17$ of float noise; z-scoring divides by it, and at $\tau=0.25$ that became a 0.9997 / 0.0003 allocation. CSCV blocks produce near-identical Sharpes routinely, so G9's temperature sweep would have been noise wherever the grid agrees
ruff B905	$8 \times \text{zip}()$ without <code>strict=</code>	Silent truncation on a length mismatch — a wrong statistic and no exception
mypy --strict	18 typing gaps, 11 of them bare dict returns	Drove the switch to frozen dataclasses (B7): one state per result object, which is what makes the twin-engine comparison meaningful
CI	Unresolvable action tag setup-uv@v9	A run that dies in 3 s having executed no step. Nothing in the repo could detect it, so a resolver now runs as the first check
CI	Local-green, CI-red property tests	hypothesis switches to a derandomised <i>ci</i> profile, drawing a different sequence. Fixed by constructing valid inputs instead of filtering for them

Decisions taken

Decision	Rationale
diff = 0.00e+00 is the single standard	Experiment A is judged against $E[\max z]$ computed by quadrature and checked to machine precision against $1/\sqrt{\pi}$ and $3/(2\sqrt{\pi})$, not against the two-term Gumbel approximation. The empirical mean estimates the truth; the formula only approximates it. Comparing Monte Carlo to the approximation conflated the two and forced an arbitrary small-N exclusion. Against exact truth it holds at every N from 2 to 10■, worst deviation 1.98 SE.
Accumulated error bands retired	Trialled and withdrawn on evidence. The pooled statistics behaved as theory predicts ($\text{rms}_z = 1.0959$ in $[0.4117, 1.5883]$) and the band did discriminate, driving a biased reference to $\text{rms}_z = 20.83$. But $\text{mean} \text{diff} $ came out at $8.87\text{e-}04$ — below the proposed lower bound of 0.0025 — and it scales as $1/\sqrt{\text{reps}}$, so any fixed bound on it constrains compute budget rather than correctness. Recoverable: <code>git checkout 37d4dac -- falsify/accumulation.py</code>
Monte Carlo reduced, never removed	Still 3 of the 5 Experiment A tests and $\sim 2.6\text{M}$ draws — more sampling than before the fix. An exact reference says where the truth is; only sampling says whether the estimator finds it.
02 Part A4 vs A1 — ruled: A1 stands	A4 asserts G1 must catch LeakyOracle, <code>sign(diff(close))</code> . It must not, because that strategy does not leak. <code>close[t]</code> lies inside <code>bars[0:t+1]</code> , which A1 permits, and every Part D convention lags the weight at least one bar, so the weight earning return <i>t</i> was decided from strictly older bars. Measured through the engine it earns +0.054 annualised Sharpe against buy-and-hold's +0.372; a strategy that genuinely sees one bar ahead earns +21.10. So the trap was mis-specified, not the harness, and tightening A1 to <code>bars[0:t]</code> would have failed every legitimate <code>close_to_close</code> strategy — which Part D explicitly permits — while catching nothing real. G7 now traps a true look-ahead oracle; the A4 strategy is kept as a documented non-violator so the reasoning survives.
Sharpe is asserted on both return conventions	00 Gate 0.1 states the true Sharpe is 0.30 and warns that 0.40 means μ was used instead of $\mu - \sigma^2/2$. Both are right, for different estimators: log returns give $(\mu - \sigma^2/2)/\sigma = 0.30$, simple returns give $\mu/\sigma = 0.40$, because $E[\exp(g)-1] = \mu/252$ exactly. Part E compounds simple returns, so the engine measures 0.40 and is correct to — asserting 0.30 against it would reject a working engine. Both targets are now pinned, so confusing them fails one direction or the other.
Actions pinned to exact releases	No floating majors anywhere. A project asserting byte-identical output under G10 should not let its own CI toolchain drift underneath it. A test enforces it.

Error budget of SR

The two-term Gumbel expression is what 01 Part B3 feeds the Deflated Sharpe as the benchmark SR, so its distance from exact truth is a bias in the DSR itself. The sign matters: it **overstates for every $N \geq 3$** , so SR is too strict rather than too lax and the resulting DSR is conservative. Below $N = 100$ the error exceeds 1% and should be quoted alongside any DSR computed there. All of it is asserted, not assumed.

N	2	3	5	10	50	100	1,000	10
Error in SR	-7.88%	+0.77%	+2.55%	+2.33%	+1.21%	+0.92%	+0.42%	+0.10%

The error is non-monotone at the bottom: the signed error flips sign between $N = 2$ and $N = 3$, so a naive monotone-decrease assertion would have been false. It decays monotonically from $N = 5$ upward.

What exists now

Module	Role	Tests
falsify/core/causality.py	G1 harness, two cut modes	14
falsify/core/event.py	Reference engine — hard-sliced prefixes, $O(T^2)$	33
falsify/core/vectorized.py	Product engine — rolling ops, exact equity loop	(shared)
falsify/core/types.py	Bars, Result, slice, InsufficientHistory — all frozen	(shared)
falsify/core/conventions.py	Part D's three conventions as (price, lag)	(shared)
falsify/costs.py	Commission, spread, slippage, borrow, cash yield	(shared)
falsify/selection.py	ArgMax, Softmax, EqualWeight, TopK	52
falsify/strategies/base.py	Strategy ABC — target weights, never orders (B4)	(shared)
tests/gates/test_prop.py	Gate 0.0 — Experiments A, B, C + the figure	14
scripts/check_action_refs.py	Resolves every workflow uses: ref	4

Next session — G6, and the first network call

The certified offline core is complete, which means the ordering rule in 00 has been satisfied: the engine has recovered known parameters from synthetic data with known truth, so real market data is now allowed to enter the system. Everything to this point was built without a single network call.

G6 is null calibration — a thousand coin-flip strategies through the whole pipeline, their Sharpe distribution forming the empirical null that the real result must beat. The compute is trivial; 03 Part C is explicit that the one genuinely hard part is **turnover matching**. A random strategy that flips every bar has enormous turnover and gets destroyed by costs, which would make the real strategy look good for entirely the wrong reason. The null has to trade at the same rate as the thing it is testing — a Markov chain whose transition probability is tuned until realised turnover matches within 5%, asserted in the test.

What G5 already tells us about G6

Break-even cost is 27.03 bps per turn at 68.8 turns a year, and a rule trading 135 times a year is unprofitable even for free. So turnover matching is not a formality here — it is the difference between a calibrated null and a flattering one, and the numbers to match against already exist.

Generated 2026-08-08 from 92e0748 · every number measured on this tree, not recalled · regenerate with: `uv run --group docs python scripts/status_report.py`